```
%macro io 4                                                    ; Macro for syscall with four
parameters
        mov rax,%1                                             ; System call number
        mov rdi,%2                                             ; First argument (file
descriptor)
        mov rsi,%3                                                     ; Second
argument (buffer)
        mov rdx,%4                                             ; Third argument (size)
        syscall                                                ; Invoke system call
%endmacro

%macro exit 0                                                  ; Macro to exit the program
        mov rax,60                                             ; syscall number for exit
        mov rdi,0                                              ; Exit code 0
        syscall                                                ; Invoke system call
%endmacro

section .data                                                  ; Data section
   msg1 db "Write an x86/64 ALP to accept 5 hexadecimal numbers from user and store them in an
array
   and display the accepted numbers",10, "Name:- Sumaanyu",10,
   "Roll:- 7256",10,"Date of Performance:- 20/01/2025",
   10                                                          ; Instruction and metadata
message
   msg1len equ $-msg1                                          ; Length of msg1
   msg2 db "Enter 5 64bit hexadecimal numbers (0-9,A-F only): ", 10   ; Prompt for user input
   msg2len equ $-msg2                                          ; Length of msg2
   msg3 db "5 64bit hexadecimal numbers are: ", 10  ; Message before displaying stored numbers
   msg3len equ $-msg3                                          ; Length of msg3
   newline db 10                                               ; Newline character
   errMsg db "You entered a wrong hexadecimal number"          ; Error message
   errLen equ $-errMsg                                         ; Length of error message

section .bss                                                   ; Uninitialized data section
   asciinum resb 17                                            ; Buffer to store ASCII input
   hexnum resq 5                                               ; Array to store 5
hexadecimal numbers

section .text                                                  ; Code section
   global _start                                               ; Entry point
   _start:
      io 1,1,msg1,msg1len                                      ; Print msg1
      io 1,1,msg2,msg2len                                      ; Print msg2 (prompt for
input)
      mov rcx,5                                                ; Set loop counter to 5
      mov rsi,hexnum                                           ; Load hexnum array
address into rsi
      next1:
         push rsi                                              ; Save rsi
         push rcx                                              ; Save rcx
         io 0,0,asciinum,17                                    ; Read input into asciinum
         call ascii_hex64                                      ; Convert ASCII to hex
```

```
        pop rcx                                 ; Restore rcx
        pop rsi                                 ; Restore rsi
        mov [rsi],rbx                           ; Store converted hex
number
        add rsi,8                               ; Move to next position in
array
        loop next1                              ; Repeat 5 times

      io 1,1,msg3,msg3len                       ; Print msg3
      mov rsi,hexnum                            ; Load hexnum array
address into rsi
      mov rcx,5                                 ; Set loop counter to 5
      next2:
        mov rbx,[rsi]                           ; Load number from array
        push rsi                                ; Save rsi
        push rcx                                ; Save rcx
        call hex_ascii64                        ; Convert and print
        pop rcx                                 ; Restore rcx
        pop rsi                                 ; Restore rsi
        add rsi,8                               ; Move to next position
        loop next2                              ; Repeat 5 times

      exit                                      ; Exit program

   ascii_hex64:
      mov rsi, asciinum                         ; Load input buffer
      mov rbx,0                                 ; Clear rbx
      mov rcx,16                                ; Set loop count to 16
      next3:
        rol rbx,4                                       ; Rotate left 4
bits
        mov al,[rsi]                            ; Get next character
        cmp al,29h                              ; Check if invalid hex
        jbe err                                 ; Jump if invalid
        cmp al,40h                              ; Check invalid range
        je err                                  ; Jump if invalid
        cmp al,67h                              ; Check invalid range
        jge err                                 ; Jump if invalid
        cmp al,47h                              ; Check further
        jge checkfurther                        ; Continue checking
        jmp operations                          ; Jump to conversion
        checkfurther:
          cmp al,60h                            ; Check invalid range
          jbe err                               ; Jump if invalid
        operations:
          cmp al,39h                            ; Check if number
          jbe sub30h                            ; Convert number
          cmp al,46h                            ; Check if letter
          jbe sub7h                             ; Convert letter
          sub al,20h                            ; Adjust ASCII value
          sub7h:
            sub al,7h                           ; Adjust ASCII value for A-F
```

```
            sub30h:
                sub al,30h                  ; Convert ASCII to hex
            jmp skip                        ; Skip error handling
        err:
            io 1,1,errMsg,errLen            ; Print error message
            io 1,1,newline,1                ; Print newline
            exit                            ; Exit program
        skip:
            add bl,al                       ; Store converted hex digit
            inc rsi                         ; Move to next character
            loop next3                      ; Repeat for 16 characters
    ret                                     ; Return from function

  hex_ascii64:
    mov rsi,asciinum                        ; Load buffer to store ASCII
output
    mov rcx,16                              ; Set loop count to 16
    next4:
        rol rbx,4                           ; Rotate left by 4 bits
        mov al,bl                                   ; Copy lower
nibble
        and al,0fh                          ; Mask lower nibble
        cmp al,9                                    ; Compare with 9
        jbe add30h                          ; Convert if <= 9
        add al,7h                           ; Adjust for A-F
        add30h:
            add al,30h                      ; Convert to ASCII
            mov [rsi],al                    ; Store character
            inc rsi                         ; Move to next position
            loop next4                      ; Repeat for 16 digits

    io 1,1,asciinum,16                      ; Print converted number
    io 1,1,newline,1                        ; Print newline
    ret                                     ; Return from function
```